

MiniSpamFilter: SMS Spam Detection Using Naive Bayes in C++

Abstract

We built a small spam filter for SMS messages in C++. It learns from labeled messages (spam or ham), estimates how likely each word is to appear in each class, and uses a Naive Bayes classifier to label new messages. We compared two feature sets, unigrams and bigrams, on two datasets: the 5574-message UCI SMS Spam Collection and a hand-written set of 40 messages. Models were trained on 80% of the data and tested on the remaining 20%, and scored with precision, recall, accuracy and F1. The project also gave us practical experience with object-oriented design in C++.

1. Introduction

Unwanted messages waste time and can be harmful (scams, fake prizes, phishing links). A spam filter has to separate them from genuine messages, also marking a real message as spam can hide something important, while letting a spam through is only an annoyance.

Objectives

- Build a working spam classifier from scratch in C++ (no machine-learning libraries).
- Understand and implement Naive Bayes, including Laplace smoothing and log-probabilities.
- Compare unigram and bigram features on a small and a large dataset.
- Evaluate the model properly using precision, recall, accuracy and F1.
- Structure the code with clean object-oriented design.

2. Background

2.1 Naive Bayes classifier

For a message made of tokens $w_1 \dots w_n$, Bayes' theorem gives the probability of each class:

$P(\text{class} \mid \text{message})$ is proportional to $P(\text{class}) \times P(w_1 \mid \text{class}) \times P(w_2 \mid \text{class}) \times \dots \times P(w_n \mid \text{class})$

The factor $P(\text{message})$ that completes the theorem is the same for both classes, so it can be dropped when we only want to know which class scores higher. The method is called "naive" because it assumes the tokens are independent of each other given the class. This is not true in real language, but the classifier often works well anyway.

2.2 Laplace smoothing

If a word never appeared in spam during training, its spam probability would be zero and would wipe out the whole product. Laplace smoothing adds 1 to every count:

$$P(\text{word} \mid \text{class}) = (\text{count of word in class} + 1) / (\text{total tokens in class} + \text{vocabulary size})$$

Adding the vocabulary size to the denominator keeps the probabilities adding up to 1.

2.3 Log-probabilities

Multiplying many small probabilities underflows to zero in floating-point numbers. Taking logs turns the product into a sum, and because log is increasing, the class with the larger score is still the same:

$$\text{score}(\text{class}) = \log P(\text{class}) + \text{sum of } \log P(\text{token} \mid \text{class})$$

2.4 Unigram and bigram features

A unigram model treats every word separately. A bigram model also adds every pair of neighbouring words (for example "hello_there"), so it can capture a little word order and common phrases. The cost is a much larger vocabulary in which most bigrams occur only once or never.

2.5 Evaluation metrics

We treat "spam" as the positive class.

	predicted spam	predicted ham
actually spam	true positive (tp)	false negative (fn)
actually ham	false positive (fp)	true negative (tn)

- **Precision** = $tp / (tp + fp + 0.001)$. Of everything declared spam, the fraction that really was spam. This is the key metric for us, because a low value means real messages are being hidden.
- **Recall** = $tp / (tp + fn + 0.001)$. Of all the real spam, the fraction we caught.
- **Accuracy** = $(tp + tn + 0.001) / \text{total}$. The fraction of all messages classified correctly. It can be misleading on imbalanced data.
- **F1** = $2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall} + 0.001)$. One number balancing precision and recall.
- we added 0.001 in denominator everywhere to make sure it isn't zero

3. Dataset

UCI SMS Spam Collection. 5574 SMS messages labeled spam or ham. Source: <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>

data.txt. A hand-written set of 40 messages, half spam and half ham, used for quick testing.

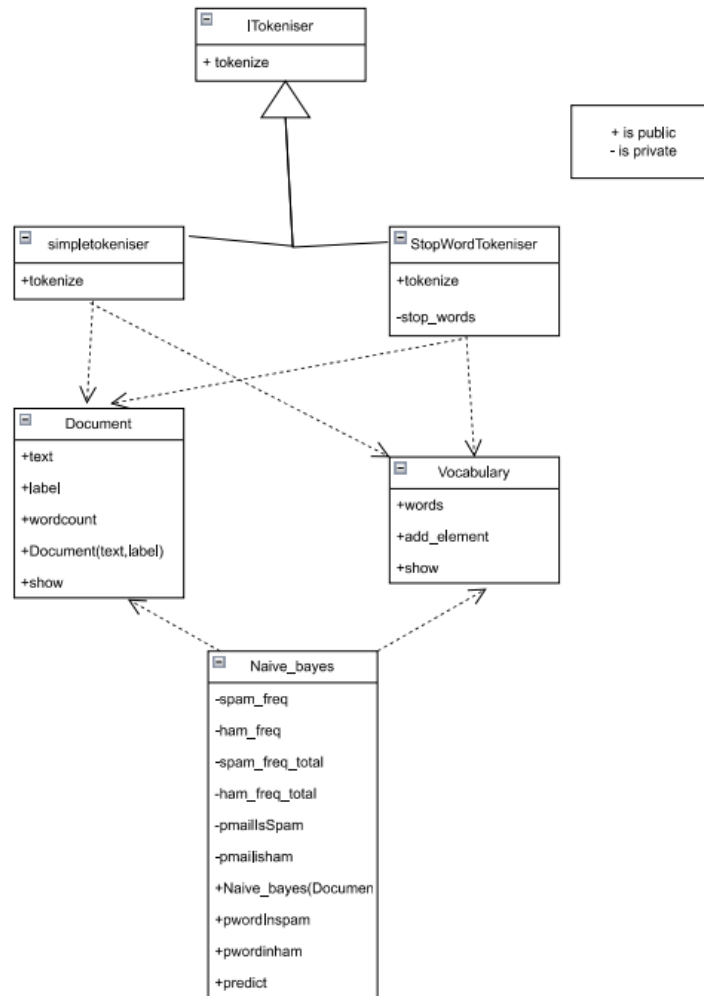
File format : The first word is the label and must be spam or ham. Anything that isn't exactly spam is counted as ham.

Preprocessing. Each message is split on whitespace, punctuation is removed, and everything is lowercased (Free!! becomes free). Stop words (the, is, at, which, and, a, an, in, to, it, of, for, on, with) are removed. For bigram runs, pairs of neighbouring words are then added to the token list.

4. System Design

The program reads the dataset, converts each line into a Document of tokens, shuffles and splits the documents, trains a Naive_Bayes model on the training part, and predicts the label of each test document.

```
dataset file -> tokeniser -> Document + Vocabulary ->  
shuffle and 80-20 split -> Naive_Bayes (train) -> predict  
test documents -> metrics
```



Relationships shown in the diagram

- SimpleTokeniser and StopWordTokeniser inherit from the abstract class ITokeniser.
- Both tokenisers depend on Document, Vocabulary (it isn't an association since nothing is stored and it is used temporarily).
- Naive_Bayes depends on Document (its training input) and on the global Vocabulary (for the vocabulary size).

5. Implementation

A worked example. This line:

```
spam Click here to claim your prize now!
```

becomes:

label : spam
tokens : "click" "here" "claim" "your" "prize" "now"
("to" is a stop word, so it is dropped)
bigrams : "click_here" "here_claim" "claim_your"
"your_prize" "prize_now"
document : each of the 11 tokens above counted once

The flow of the program

- `ucl_sms_dataset.txt->load_training_file()` [giving txt file to the loading function]
- `tokeniser.tokenize(line)` [every word of a line of the file is cleaned , bigrams are taken too , and every unigram & bigram string is put into a vector of strings]
- Document creation [from the vector of strings , the first element(label) is separated from the text , converted into a Document object having variables: `text(vector<string>)` and `label(string)`]
- Data splitting [By now , the `all_docs` (global vector of all document objs) is populated , so we shuffle and perform 80/20 split , 80% for training , 20% for testing]
- `Naïve_Bayes(train_docs)` [we pass the vector of docs containing training docs to the Naïve Bayes object which extracts info (trains) from the `train_docs` which will be used for the prediction formulae]
- `predict()` [based on the training , the model is used to predict labels for the remaining 20% data]
- Model metrics [based on the testing , precision , accuracy , recall , f1 score are calculated]
-

main.cpp

- `main()`: the entry point. It loads the dataset, shuffles it, splits it 80-20, trains Naive_Bayes on the 80%, tests it on the 20%, prints the metrics, and then classifies a message typed by the user.
- **all_docs and v1**: two global variables defined here. `all_docs` is a vector holding one Document per message, and `v1` is the Vocabulary of every token seen. They are declared extern in `tokeniser.h` so the other files can use the same copies.
- **Evaluation code**: counts tp, fp, tn and fn, where "positive" means "spam". From these it computes precision, recall, accuracy and F1. A tiny 0.001 is added to the denominators to avoid dividing by zero.

file_io.h

- `has_content(line)`: checks that a line still has at least one word after punctuation is removed. It stops blank or punctuation-only lines from crashing the tokeniser.
- `load_training_file(filename, tokeniser)`: reads the file line by line, skips unusable lines, handles Windows line endings, and passes each line to the tokeniser. It returns how many lines were loaded and throws a `runtime_error` if the file can't be opened.

tokeniser.h / tokeniser.cpp

- `cleanedword(word)`: removes all punctuation from a word and lowercases it, so `Free!!` becomes `free`.
- `bigram_maker(tokens)`: appends every pair of neighbouring words, joined with `_`, to the same list. For example `["win", "free", "prize"]` becomes `["win", "free", "prize", "win_free", "free_prize"]`. This lets the model see a little word order.
- **Document**: one message. It stores the label (spam or ham), the text (its list of tokens) and `wordCount` (how many times each token appears in that message). The constructor builds `wordCount` and skips empty tokens.
- **Vocabulary**: a set of every distinct token seen across all messages. `add_element()` inserts the tokens of one message, and `show()` prints them all. Naive_Bayes uses its size for Laplace smoothing.
- **ITokeniser**: an abstract class with one function, `tokenize(text)`. Both tokenisers implement it, so the rest of the code doesn't care which one is used.
- `SimpleTokeniser::tokenize()`: splits a line on whitespace, cleans each word, takes the first word as the label, adds bigrams, then builds a Document and stores it in `all_docs` and `v1`. (converts "sujit sir is best" to `["sujit", "sir", "is", "best"]`)
- `StopWordTokeniser::tokenize()`: does the same, but first drops empty tokens and stop words (the, is, at, which, and, a, an, in, to, it, of, for, on, with), which carry almost no signal for spam detection. It returns the token list, so a typed message can go through the same steps before prediction.

naive_bayes.h / naive_bayes.cpp

- **Private data:** spam_freq and ham_freq store how many times each token appears in all spam and all ham messages. spam_freq_total and ham_freq_total store the total token counts for each class. p_maillSpam and p_maillHam store the fraction of messages that are spam and ham. These are stored as private since they are crucial parts of the prediction process and we do not want to risk these variables accidentally getting changed, since that would ruin the entire prediction process (defensive programming).
- **Naive_Bayes(docs) (constructor):** prepares for training by finding extracting data for the spam_freq map, ham_freq map and finding out p_maillSpam and p_maillHam. Goes through every document from the list of documents provided to do so.
- **p_wordInSpam(word) and p_wordInHam(word):** It basically means, if we look into all the documents of this class, then what's the probability of finding the exact set of words found in the input document. It returns the Laplace-smoothed probability of a word in each class, $(\text{count} + 1) / (\text{class total} + \text{vocabulary size})$. The +1 stops an unseen word from making the whole probability zero. The vocabulary size in the denominator is for ensuring that all probabilities sum up to 1.
- **predict(tokens):** Discards the denominator from the general formula of naïve bayes, only utilises the numerator. It takes the log on both sides to ensure values don't shrink too much. Adds up log of p_wordInSpam() for every word to the p_maillSpam to find out the spam score for a new document, similarly finds out the ham score. Predicts the class with the higher score. When scores are same, it predicts Ham to prevent taking the risk of classifying an important email as Spam.x

6. Results

Features	Dataset	Precision	Recall	Accuracy	F1
Unigram	data.txt (40)	83.3195%	99.98%	97.4891%	90.8926%
Bigram	data.txt (40)	99.98%	99.98%	99.9875%	99.98%
Unigram	UCI (5574)	90.9085%	94.5939%	98.0268%	92.7146%
Bigram	UCI (5574)	86.0601%	95.9453%	97.399%	90.7342%

Observations

- On the UCI dataset, unigram scored higher than bigram on the metrics we measured.

- On data.txt, bigram scored close to 100% on all metrics.
- All runs classified the sample message we tried correctly as spam or ham.

Results:

- for unigram (with dataset.txt - only 40):

```

PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
Loaded 40 documents.
MODEL METRICS -
Precision: 83.3195%
Recall:99.98%
Accuracy: 87.4891%
F1 Score:90.8926%
enter email u want to see classified
hey there good to see you
hey there good to see you is predicted as ham
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
Loaded 40 documents.
MODEL METRICS -
Precision: 83.3195%
Recall:99.98%
Accuracy: 87.4891%
F1 Score:90.8926%
enter email u want to see classified
congratulations u won a car for 10 rupees
congratulations u won a car for 10 rupees is predicted as spam
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7>

```

- for unigram (with uci sms dataset.txt - has 5574):

```

PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> g++ main.cpp tokeniser.cpp naive_bayes.cpp -o test
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
Loaded 5574 documents.
MODEL METRICS -
Precision: 90.9085%
Recall:94.5939%
Accuracy: 98.0268%
F1 Score:92.7146%
enter email u want to see classified
hey there good to see you
hey there good to see you is predicted as ham
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
Loaded 5574 documents.
MODEL METRICS -
Precision: 90.9085%
Recall:94.5939%
Accuracy: 98.0268%
F1 Score:92.7146%
enter email u want to see classified
congratulations u won a car for 10 rupees
congratulations u won a car for 10 rupees is predicted as spam
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7>

```

- for bigram (with dataset.txt - only 40):

```

● PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> g++ main.cpp tokeniser.cpp naive_bayes.cpp -o test
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
● Loaded 40 documents.
MODEL METRICS -
Precision: 99.98%
Recall:99.98%
Accuracy: 99.9875%
F1 Score:99.98%
enter email u want to see classified
hey there good to see you
hey there good to see you is predicted as ham
● PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
Loaded 40 documents.
MODEL METRICS -
Precision: 99.98%
Recall:99.98%
Accuracy: 99.9875%
F1 Score:99.98%
enter email u want to see classified
congratulations u won a car for 10 rupees
congratulations u won a car for 10 rupees is predicted as spam
○ PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7>

```

- for bigram(with uci_sms_dataset.txt - has 5574):

```

● PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> g++ main.cpp tokeniser.cpp naive_bayes.cpp -o test
PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
● Loaded uci_sms_dataset.txtLoaded 5574 documents.
MODEL METRICS -
Precision: 86.0601%
Recall:95.9453%
Accuracy: 97.399%
F1 Score:90.7342%
enter email u want to see classified
hey there good to see you
hey there good to see you is predicted as ham
● PS C:\Users\srika\Downloads\cpp_proj_proto7zip\cpp_proj_proto-7> ./test
Loaded uci_sms_dataset.txtLoaded 5574 documents.
MODEL METRICS -
Precision: 86.0601%
Recall:95.9453%
Accuracy: 97.399%
F1 Score:90.7342%
enter email u want to see classified
congratulations u won a car for 10 rupees
congratulations u won a car for 10 rupees is predicted as spam

```

all of them predict the message correctly as spam or ham but they vary in metrics by a good amount, for the uci_sms_dataset, the metrics for unigram>bigram, the reason being that the emails here are sparse and bigrams dont work well with sparse data, unigram is a better fit.

However, coming to the dataset.txt which is relatively small, bigram shows almost 100% in all metrics, which isnt possible for a realistic model, the reason here is that bigram overfit onto the data, and the test data is very similar to training data.(since its a small sized dataset), Unigram even tho its less in metrics, it can be more trusted since it likely hasnt overfit and can predict test data even if they arent very similar.

Only about 13% of the UCI messages are spam, so a model that always answered "ham" would already reach about 87% accuracy. This is why precision and recall are reported alongside accuracy.

7. How the project evolved (versions)

- 1 taking inputs from terminal for train and testing, unigrams only, everything in cpp files.
- 2 adding file_io and taking input from data.txt
- 3 using a bigger dataset, the uci_sms_dataset.txt
- 4 seeing how good bigrams are compared to unigrams by implementing them, however showing that unigrams are better in this case.

8. Conclusion

We built a complete spam classifier in C++ without any machine-learning library, covering file reading, text cleaning, tokenising, Naive Bayes training and prediction, and evaluation. The classifier labels messages using class fractions and word probabilities combined through log-scores, with Laplace smoothing handling unseen words.

What the experiments showed. On the 5574-message UCI dataset, single-word features worked better than adding bigrams. On the 40-message set, bigrams looked almost perfect, but the test set was so small and repetitive that the result says little about real performance. Overall, adding more features is not automatically an improvement. How useful a feature is depends on how much data there is to support it.

What we learned

- How Naive Bayes works in practice. The ideas of priors, conditional probabilities, smoothing and log-scores became concrete once we had to implement each one and see what happens when a word has never been seen.
- Evaluation needs care. Accuracy alone is misleading when one class dominates. Precision matters most for a spam filter, because falsely flagging a real message is the costly mistake. We also saw that a tiny test set produces numbers that look impressive but can't be trusted, and that one random split can look better or worse than average by luck.
- Preprocessing must match between training and prediction. Moving the cleaning and stop-word removal into the tokenisers, instead of repeating it inside predict, made sure training data and typed messages are treated the same way.
- Good design saves effort. Putting the tokenisers behind an ITokeniser interface let us switch between unigram and stop-word versions without touching the rest of the code. Splitting the code into header and source files kept it readable and quicker to rebuild.
- Choosing the right dataset for your model is also important since we saw that data.txt had barely any data and bigrams overfitted on it, whereas the uci_sms_dataset.txt was relatively better.